

Lesson 13 · AI-Assisted Testing

PDF: [this lesson](#) · **Install (Linux · macOS · Windows):** [guide](#) · [PDF](#)

Part of [local-ai-lab](#) - a hands-on course for building local AI.

Course home: <https://nikolareljin.github.io/local-ai-lab/> **Source:** <https://github.com/nikolareljin/local-ai-lab>

Lessons: [1 · RAG](#) → [2 · MCP](#) → [3 · Hybrid retrieval](#) → [4 · RAG safety](#) → [5 · RAG evaluation](#) → [6 · Repo assistant](#) → [7 · LangChain](#) → [8 · LangGraph](#) → [9 · Ollama tools](#) → [10 · Semantic Kernel](#) → [11 · Bedrock](#) → [12 · Google ADK](#) → **13 · AI-assisted testing (you are here)** → [14 · AI code review](#) → [15 · Docs from changes](#)

Status: planned. Outline below; full step-by-step coming later. Star the repo to follow along.

The idea

Every lesson in this course ships a test, and you have been reading them all along. This lesson turns the model on the test suite itself: generate tests from a change, run them, read the failures, and let the failure guide the fix.

The uncomfortable part first, because it is the whole lesson: **a generated test that passes is worthless**. It tells you the code does what it currently does. What you want is a test that **fails for the right reason** before you fix anything - and a model asked to "write tests for this file" will almost never give you one, because it reads the implementation and describes it back to you.

What you'll learn

- **Generate from a diff, not a repo** - the change is the specification; the rest of the file is noise
- **Fail first** - make the model write the test against the **intended** behaviour, run it red, then fix
- **Judge a suite by what it catches** - break a line on purpose and see whether anything goes red. A suite that survives a deliberate bug is decoration, however many tests it has
- **Keep the model out of the assertion** - it proposes, you approve. An assertion nobody read is a future false green
- **Where it genuinely wins** - edge cases, boundary values, and the table-driven cases people skip

Builds on

Concept	From
golden sets and regression gates	Lesson 5
indexing a repo into cited passages	Lesson 6
shipping it as a callable tool	Lesson 2
generated tests, and what they are worth	Lesson 13 (this one)

Prerequisites

[Lesson 1](#). Lessons 5 and 6 help: a generated suite needs the same regression discipline as a golden set, and the generator needs to read your repo before it can test it.

Next lesson

[Lesson 14 · AI Code Review & Issue Detection →](#)

Course: nikolareljin.github.io/local-ai-lab · **Author:** [Nik Reljin](#)